Rust Lifetimes

A field guide to regions, elision and variance —
with a close reading of `'_` and `'static`

Every program in this document was compiled with `rustc 1.75.0`, edition 2021. Compiler diagnostics are quoted verbatim; passing and failing examples are marked.

In one page

- A lifetime is a *compile-time region* (a set of program points), never a runtime value. Annotations *describe* relationships between borrows; they cannot make anything live longer.
- `'_` is not a lifetime name. It means "some lifetime I don't need to name": fresh for each parameter, resolved by the elision rules in a return type, and inferred in a body. Its three real jobs are (i) exposing hidden lifetime parameters, (ii) `impl`-headers, and (iii) opting trait objects and `impl Trait` into borrowing (`dyn Trait + '_`).
- `'static` has *two* meanings: `&'static T` (a reference valid for the rest of the program) and `T: 'static` (a *bound*: "T holds no borrow that can expire"). The bound is far more common, and it does **not** mean the value lives forever: a local `String` is `'static`.
- Default trait-object bounds are the hidden source of most `'static` errors: `Box<dyn Trait>` means `Box<dyn Trait + 'static>`, whereas `&'a dyn Trait` means `&'a (dyn Trait + 'a)`.

Contents

1 The mental model: regions, not durations 2

2 Types and methods: whose borrow is it? 4

3 Lifetime elision 6

4 Subtyping and variance 15

5 The anonymous lifetime `'_` 17

6 The `'static` lifetime 21

7 Higher-ranked bounds: `for<'a>` 29

8 Error catalogue 30

9 Quick reference 31

A Verification log 32

B Further reading 33

1 The mental model: regions, not durations

For the borrow checker, a lifetime `'a` is a *region*: a set of points in the control-flow graph of a function body. A reference `&'a T` is a promise that the referent stays valid, and is not accessed in a conflicting way, at every point of `'a`. Three consequences shape everything else in this guide.

1. **Lifetimes are erased.** They have no runtime representation and do not influence code generation; they exist only to give the checker a system of constraints to solve.
2. **They follow use, not scope.** Since non-lexical lifetimes (NLL), a borrow is live from its creation up to the last point where a reference derived from it *may still be used*, not to the closing brace of the enclosing block.
3. **Only signatures carry lifetimes you write.** Inside a body the compiler infers every region on its own; at function boundaries, annotations (or their elided defaults) are the contract.

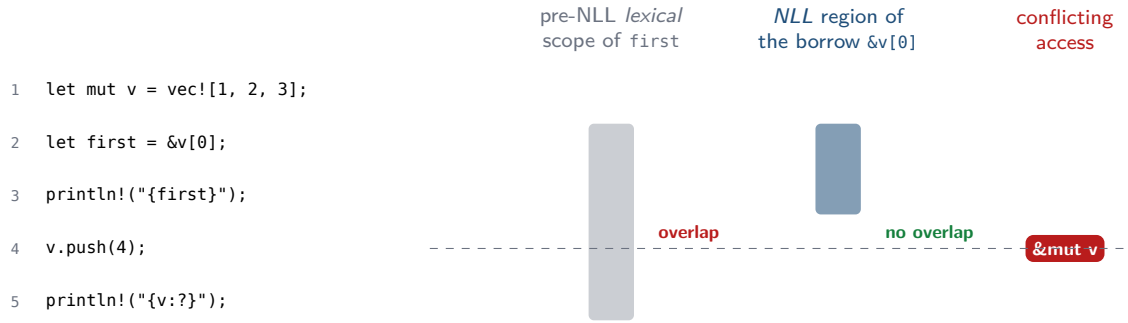


Figure 1: Why Listing 1 compiles. The borrow `&v[0]` is live only until its last use (line 3), so the `v.push` on line 4 does not intersect it. A purely lexical rule would keep it alive to the end of the block.

Listing 1 NLL: the borrow ends at its last use ✓ compiles

ok_02_nll.rs

```rust
fn main() {
    let mut v = vec![1, 2, 3];
    let first = &v[0];
    println!("{first}"); // last use of `first`: its lifetime ends here
    v.push(4);           // so this mutable borrow is accepted (NLL)
    println!("{v:?}");
}
```

```
1
[1, 2, 3, 4]
```

Listing 2 Move the later use after the `push` and the borrow is live across it ✗ rejected

fail_02_nll.rs

```rust
fn main() {
    let mut v = vec![1, 2, 3];
    let first = &v[0];
    v.push(4);           // needs &mut v while `first` is still live
    println!("{first}"); // <- this later use keeps the shared borrow alive
}
```

```
error[E0502]: cannot borrow `v` as mutable because it is also borrowed as immutable
 --> fail_02_nll.rs:4:5
  |
3 |     let first = &v[0];
  |                 - immutable borrow occurs here
4 |     v.push(4);           // needs &mut v while `first` is still live
  |     ^^^^^^^^ mutable borrow occurs here
5 |     println!("{first}"); // <- this later use keeps the shared borrow alive
  |              ------- immutable borrow later used here
```

1.1 Annotations describe; they never extend

Listing 3 The classic `longest` ✓ compiles

ok_01_longest.rs

```rust
fn longest<'a>(x: &'a str, y: &'a str) -> &'a str {
    if x.len() >= y.len() { x } else { y }
}

fn main() {
    let outer = String::from("a longer string");
    let result;
    {
        let inner = String::from("short");
        result = longest(outer.as_str(), inner.as_str());
        println!("{result}"); // fine: `inner` is still alive here
    }
}
```

In `fn longest<'a>(x: &'a str, y: &'a str) -> &'a str` the single name `'a` says: "the result is valid only where *both* inputs are", i.e. for the intersection of their regions. The body is checked against this signature (returning `x` or `y` is justified), and every caller is checked against it *and nothing else*. This modularity is why the defaults chosen by elision (Section 3) matter so much: they *are* the contract.

Listing 4 Use the result after one input died ✗ rejected

fail_01_dangling.rs

```rust
fn longest<'a>(x: &'a str, y: &'a str) -> &'a str {
    if x.len() >= y.len() { x } else { y }
}

fn main() {
    let outer = String::from("a longer string");
    let result;
    {
        let inner = String::from("short");
        result = longest(outer.as_str(), inner.as_str());
    } // `inner` dropped here ...
    println!("{result}"); // ... but `result` may point into it
}
```

```
error[E0597]: `inner` does not live long enough
  --> fail_01_dangling.rs:10:42
   |
9  |         let inner = String::from("short");
   |             ----- binding `inner` declared here
10 |         result = longest(outer.as_str(), inner.as_str());
   |                                          ^^^^^ borrowed value does not live long enough
11 |     } // `inner` dropped here ...
   |     - `inner` dropped here while still borrowed
12 |     println!("{result}"); // ... but `result` may point into it
   |               -------- borrow later used here
```

Here `'a` must cover the final `println!`, because `result` is used there; but `&inner` is valid only inside the inner block. Nothing in the *body* of `longest` is wrong; the contradiction is between the call site and the contract.

1.2 Notation

Syntax	Read as	Meaning
`'a`	"lifetime a"	A region variable.
`&'a T`		Reference to a `T`, valid throughout `'a`.
`'a: 'b`	"`'a` outlives `'b`"	Region `'a` contains every point of `'b`.
`T: 'a`	"`T` outlives `'a`"	Every lifetime occurring inside `T` outlives `'a`: values of type `T` stay valid throughout `'a`.
`'static`		The region covering the whole program; it outlives every other lifetime.
`'_`	"anonymous lifetime"	Placeholder; see Section 5.
`for<'a> ...`	"for all `'a`"	Higher-ranked quantification; see Section 7.

Writing `&'a T` also *implies* `T: 'a` (the type must be well-formed); this is why you almost never write that bound by hand.

2 Types and methods: whose borrow is it?

A struct that stores a reference needs a lifetime parameter, and that parameter is part of the type: `Parser<'x>` and `Parser<'y>` are different types, related by variance (Section 4). Inside `impl<'src> Parser<'src>`, the name `'src` denotes the lifetime of the *data the parser reads*. Every method additionally gets its own anonymous lifetime for `&self` or `&mut self`. A method returning a reference therefore has a genuine choice:

Return type	Result borrows from	Effect on the caller
`Option<&'src str>`	the *text*	parser is free again; results can be kept across calls
`Option<&str>` (elided)	the *`&mut self` borrow*	parser stays exclusively borrowed while any result lives

Elision always picks the second row (rule 3, Section 3). That is correct for something like `fn buf(&mut self) -> &mut [u8]`, but wrong for tokenisers, iterators and cursors:

Listing 5 Returning `'src` instead of the `&mut self` lifetime ✓ compiles

ok_03_parser.rs

```rust
use std::fmt;

struct Parser<'src> {
    input: &'src str,
    pos: usize,
}

impl<'src> Parser<'src> {
    fn new(input: &'src str) -> Self {
        Parser { input, pos: 0 }
    }

    /// The result borrows from the *text* ('src), not from the `&mut self`
    /// borrow, so earlier results stay usable across later calls.
    fn next_word(&mut self) -> Option<&'src str> {
        let input: &'src str = self.input;
        let rest = &input[self.pos..];
        let trimmed = rest.trim_start();
        if trimmed.is_empty() {
            return None;
        }
        let start = self.pos + (rest.len() - trimmed.len());
        let end = trimmed
            .find(char::is_whitespace)
            .map_or(input.len(), |i| start + i);
        self.pos = end;
        Some(&input[start..end])
    }
}

// `'_` : "some lifetime, I don't need to name it"
impl fmt::Display for Parser<'_> {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "Parser@{} over {:?}", self.pos, self.input)
    }
}

fn main() {
    let text = String::from("lifetimes are regions");
    let mut p = Parser::new(&text);
    let a = p.next_word();
    let b = p.next_word(); // OK: `a` does not keep `p` borrowed
    println!("{a:?} {b:?} / {p}");
}
```

```
Some("lifetimes") Some("are") / Parser@13 over "lifetimes are regions"
```

Listing 6 Same code, elided return type: results pin the parser ✗ rejected

fail_03_elided_self.rs (abridged)

```rust
struct Parser<'src> {
    input: &'src str,
    pos: usize,
}

impl<'src> Parser<'src> {
    // fn new(..) unchanged

    // Elision rule 3: the output lifetime is that of `&mut self`.
    fn next_word(&mut self) -> Option<&str> {
        // body unchanged: returns Some(&self.input[start..end])
    }
}

fn main() {
    let text = String::from("lifetimes are regions");
    let mut p = Parser::new(&text);
    let a = p.next_word();
    let b = p.next_word(); // second &mut borrow while `a` is live
    println!("{a:?} {b:?}");
}
```

```
error[E0499]: cannot borrow `p` as mutable more than once at a time
  --> fail_03_elided_self.rs:31:13
   |
30 |     let a = p.next_word();
   |             - first mutable borrow occurs here
31 |     let b = p.next_word(); // second &mut borrow while `a` is live
   |             ^ second mutable borrow occurs here
32 |     println!("{a:?} {b:?}");
   |               ----- first borrow later used here
```

Rule of thumb

If the reference you return points into data that the struct merely *holds a reference to*, name the struct's own lifetime in the return type. This is exactly why `slice::Iter<'a, T>::next` can return `Option<&'a T>` and you can collect many items while the iterator is still in use. A "lending" iterator, whose items *do* borrow from `&mut self`, cannot be expressed by `Iterator` at all.

3 Lifetime elision

Elision is a fixed, purely syntactic rewrite applied to function signatures (and to `Fn(..)` sugar and `impl` headers). It never looks at the body.

1. Each elided lifetime in the *inputs* becomes a distinct fresh lifetime parameter.
2. If there is *exactly one* input lifetime position (elided or named), that lifetime is assigned to all elided output lifetimes.
3. If there are several, but one of them is `&self` or `&mut self`, the lifetime of `self` is assigned to all elided output lifetimes.

Otherwise elision fails with E0106 ("missing lifetime specifier").

You write	The compiler reads	Rule
`fn f(x: &str) -> &str`	`fn f<'a>(x: &'a str) -> &'a str`	1, 2
`fn f(x: &str, y: &str) -> &str`	*error E0106*	1, none applies
`fn f<'a>(x: &'a str, y: &str) -> &str`	*error E0106* (two input lifetimes)	none applies
`fn m(&self, k: &str) -> &str`	`fn m<'s,'k>(&'s self, k: &'k str) -> &'s str`	1, 3
`fn f(x: &mut &str)`	`fn f<'a,'b>(x: &'a mut &'b str)` (independent)	1
`fn f(s: &str) -> Iter`	`fn f<'a>(s: &'a str) -> Iter<'a>` (hidden; prefer `Iter<'_>`)	1, 2
`fn(&str) -> &str` (pointer type)	`for<'a> fn(&'a str) -> &'a str`	HR, 1, 2
`static S: &str` / `const C: &[u8]`	`&'static str` / `&'static [u8]`	item default

3.1 Why those two rows are errors

Take the two rejected signatures from the table and apply the rules mechanically.

Case A: `fn f(x: &str, y: &str) -> &str`.

1. Rule 1 gives each elided *input* its own fresh lifetime: `fn f<'1, '2>(x: &'1 str, y: &'2 str) -> &'? str`.
2. Rule 2 applies only if there is *exactly one* input lifetime. There are two, so it is skipped.
3. Rule 3 applies only if one input is `&self` or `&mut self`. Neither is, so it is skipped.

No rule fills in `'?`, so elision fails with E0106.

Case B: `fn f<'a>(x: &'a str, y: &str) -> &str`. Rule 2 counts *every* lifetime position in the inputs, named or elided. Here there are two: the named `'a` and a fresh anonymous one for `y`. The count is therefore two, rule 2 is skipped, there is no `self`, and again nothing determines the output. Naming one input does not make it "the" input; the compiler does not guess that the named one, or the elided one, is the intended source.

No further rule can rescue these cases. Elision is a closed set of exactly the three rules above; the compiler does not fall back to a heuristic when they run out. It will not pick "the first parameter", "the longest-lived input", "the named one" or "whichever the body returns", and writing `'_` in the output does not help either, because `'_` in a return type just *requests* elision (Listing 27). Once rules 2 and 3 both fail, the only way forward is to write the output lifetime explicitly. Section 3.3 tests the most natural candidate rules and shows concretely how each one fails.

Listing 7 Three honest ways to state the contract ✓ compiles

ok_18_elision_fixes.rs

```rust
// The contract must be stated. Three honest choices:

// 1. Result may come from EITHER input: tie everything together.
fn either<'a>(x: &'a str, y: &'a str) -> &'a str {
    if x.len() > y.len() { x } else { y }
}

// 2. Result comes only from `x`: say so; `y` stays independent.
fn first_only<'a>(x: &'a str, _y: &str) -> &'a str {
    x
}

// 3. Result borrows from neither: no lifetime needed.
fn owned(x: &str, y: &str) -> String {
    format!("{x}{y}")
}

fn main() {
    let long = String::from("long-lived");
    let r;
    {
        let short = String::from("short");
        r = first_only(&long, &short); // fine: `r` does not depend on `short`
        println!("{}", either(&long, &short));
        println!("{}", owned(&long, &short));
    }
    println!("{r}");
}
```

```
long-lived
long-livedshort
long-lived
```

Listing 8 Cases A and B from the table ✗ rejected

fail_18_elision_cases.rs

```rust
// Case A: two elided inputs, elided output
fn a(x: &str, y: &str) -> &str {
    if x.len() > y.len() { x } else { y }
}

// Case B: one named + one elided input, elided output
fn b<'a>(x: &'a str, y: &str) -> &str {
    x
}

fn main() {}
```

```
error[E0106]: missing lifetime specifier
 --> fail_18_elision_cases.rs:2:27
  |
2 | fn a(x: &str, y: &str) -> &str {
  |         ----     ----     ^ expected named lifetime parameter
  |
  = help: this function's return type contains a borrowed value, but the signature does not say whether it is borrowed from `x` or `y`
help: consider introducing a named lifetime parameter
  |
2 | fn a<'a>(x: &'a str, y: &'a str) -> &'a str {
  |     ++++     ++          ++          ++

error[E0106]: missing lifetime specifier
 --> fail_18_elision_cases.rs:7:34
  |
7 | fn b<'a>(x: &'a str, y: &str) -> &str {
  |             -------     ----     ^ expected named lifetime parameter
  |
  = help: this function's return type contains a borrowed value, but the signature does not say whether it is borrowed from `x` or `y`
help: consider using the `'a` lifetime
  |
7 | fn b<'a>(x: &'a str, y: &str) -> &'a str {
  |                                   ++
```

Why the compiler refuses to guess

The compiler never inspects the body to resolve elision. In case A the body happens to return either `x` or `y`, and in case B only `x`, but the signature is the *contract* that callers are checked against. If the body could silently change the contract, editing the body (say, returning `y` instead of `x`) would break callers in other crates without touching the signature. Requiring you to write the relationship keeps the signature authoritative, and the compiler's own hint above spells out the question it cannot answer: "borrowed from `x` or `y`?"

The choice is a real design decision, not a syntax chore:

Signature	Result may borrow from	Caller consequence
`fn f<'a>(x: &'a str, y: &'a str) -> &'a str`	either input	result must not outlive *either* argument
`fn f<'a>(x: &'a str, y: &str) -> &'a str`	only `x`	`y` may die while the result lives; the body may not return `y`
`fn f(x: &str, y: &str) -> String`	neither	no borrow at all

3.2 Why a single reference in a struct cannot be elided

Strip the problem to its minimum: one struct, one field, one reference. If elision ever worked for a type, it would work here.

Listing 9 The simplest struct with a reference ✗ rejected

`fail_19_struct_single_ref.rs`

```rust
// The simplest possible case: one struct, one reference field, nothing else.
struct Holder {
    r: &str,
}

fn main() {}
```

```
error[E0106]: missing lifetime specifier
 --> fail_19_struct_single_ref.rs:3:8
  |
3 |     r: &str,
  |        ^ expected named lifetime parameter
  |
help: consider introducing a named lifetime parameter
  |
2 ~ struct Holder<'a> {
3 ~     r: &'a str,
  |
```

The error is E0106 even though there is only one reference and nothing to be ambiguous about. Writing `&'_ str` gives E0106 and writing `&'a str` without declaring `'a` gives E0261 (both verified in Appendix A, `fail_16`). The reasons are structural, not a matter of missing rules.

1. **Elision is defined over function signatures.** Its rules choose the output lifetimes from the *input* lifetimes. A struct definition has no inputs and no outputs: a field is not a parameter, so "exactly one input lifetime" (rule 2) has nothing to count. The fact that there is a single reference is irrelevant.
2. **In a struct the lifetime is part of the type, so it must be declared.** A function's fresh lifetimes (rule 1) are local to its signature and change nothing outside it. A struct field's lifetime would have to become a parameter of the struct itself: `Holder` would secretly be `Holder<'x>`. Every use of the type, every `impl`, every field of another struct, and the variance computation (Section 4) depend on that parameter, yet the declaration `struct Holder` would not show it. Rust requires the parameter to be written where it is declared, as the compiler's own suggestion shows.
3. **The only sensible implicit meaning would be `'static`, and that is a trap.** Items such as `static S: &str` do default to `'static`, because there the answer is unambiguous. For a struct field it would silently restrict the type to literals and leaked data, as the next listing shows. Rust makes you choose instead.

Listing 10 What an implicit `'static` default would cost ✗ rejected

`fail_20_static_field_local.rs`

```rust
// What a silent `'static` default for fields would cost:
struct Literal {
    r: &'static str,
}

fn main() {
    let text = String::from("hello");
    let l = Literal { r: &text }; // a borrow of a local is not 'static
    println!("{}", l.r);
}
```

```
error[E0597]: `text` does not live long enough
  --> fail_20_static_field_local.rs:8:26
   |
7  |     let text = String::from("hello");
   |         ---- binding `text` declared here
8  |     let l = Literal { r: &text }; // a borrow of a local is not 'static
   |                          ^^^^^
   |                          |
   |                          borrowed value does not live long enough
   |                          this usage requires that `text` is borrowed for `'static`
9  |     println!("{}", l.r);
10 | }
   | - `text` dropped here while still borrowed
```

Choosing is a real design decision, and each choice means something different for everyone who uses the struct:

Field	Meaning	Consequence
`struct S<'a> { r: &'a str }`	borrows from some string	`S` cannot outlive that string; `'a` appears in every use of the type
`struct S { r: String }`	owns its data	no lifetime anywhere; costs an allocation or clone
`struct S { r: &'static str }`	literals or leaked data only	no lifetime parameter, but only `'static` data fits

Listing 11 The three choices side by side ✓ compiles

ok_19_struct_fixes.rs

```rust
// Three ways to give the field a meaning. Each one is a design decision.

// 1. Borrow: the lifetime becomes a parameter of the TYPE.
struct Borrowed<'a> {
    r: &'a str,
}

// 2. Own the data: no lifetime anywhere.
struct Owned {
    r: String,
}

// 3. Literals only: an explicit 'static.
struct Literal {
    r: &'static str,
}

fn wrap<'a>(s: &'a str) -> Borrowed<'a> {
    Borrowed { r: s }
}

fn main() {
    let text = String::from("hello");
    let b = wrap(&text);                // Borrowed<'_> tied to `text`
    let o = Owned { r: text.clone() };  // independent of `text`
    let l = Literal { r: "literal" };   // fine: a literal is 'static
    println!("{} {} {}", b.r, o.r, l.r);
}
```

```
hello hello literal
```

Note the contrast with functions: once the parameter exists, *uses* of the struct can elide it again, since a function signature may mention `Borrowed` without arguments (hidden, see the `elided_lifetimes_in_paths` lint) or write `Borrowed<'_>`. Only the *definition* must be explicit.

3.3 Concrete failures of the obvious extra rules

In `longest` the right answer feels obvious because *we* have read the body. Elision may not read the body, so any extra rule must choose from the signature alone. To see why that is not enough, take a function where the correct contract is *not* the obvious one and test the three most natural candidate rules on it. (These rules are hypothetical: Rust has none of them. For each I wrote out by hand the signature it would produce and let the compiler judge it.)

The function looks a key up in a table. The result borrows from the *table*, not from the key, and the key is a temporary that may die first, which is perfectly valid:

Listing 12 The intended contract, written explicitly ✓ compiles

ok_21_lookup_explicit.rs

```rust
// The intended contract: the result borrows from `table`, not from `key`.
struct Table {
    rows: Vec<(String, String)>,
}

fn lookup<'t>(key: &str, table: &'t Table) -> Option<&'t str> {
    table.rows.iter().find(|row| row.0 == key).map(|row| row.1.as_str())
}

fn main() {
    let table = Table { rows: vec![("a".to_string(), "alpha".to_string())] };
    let hit;
    {
        let key = String::from("a"); // a temporary key ...
        hit = lookup(&key, &table);
    } // ... dies here, but the result comes from `table`
    println!("{hit:?}");
}
```

```
Some("alpha")
```

Candidate A: “tie the output to the first reference parameter”

This is the rule most people propose first, and for `longest`-like code it looks harmless. For `lookup` it yields `fn lookup<'k, 't>(key: &'k str, table: &'t Table) -> Option<&'k str>`:

Listing 13 Rule A applied to `lookup` ✗ rejected

fail_21_rule_first.rs (abridged)

```rust
// Hypothetical rule A: "tie the output to the FIRST reference parameter".
// struct Table and main(): identical to the previous listing

fn lookup<'k, 't>(key: &'k str, table: &'t Table) -> Option<&'k str> {
    table.rows.iter().find(|row| row.0 == key).map(|row| row.1.as_str())
}
```

```
error: lifetime may not live long enough
 --> fail_21_rule_first.rs:7:5
  |
6 | fn lookup<'k, 't>(key: &'k str, table: &'t Table) -> Option<&'k str> {
  |           --  -- lifetime `'t` defined here
  |           |
  |           lifetime `'k` defined here
7 |     table.rows.iter().find(|row| row.0 == key).map(|row| row.1.as_str())
  |     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ function was supposed to return data with lifetime `'k`
  ↪  but it is returning data with lifetime `'t`
  |
  = help: consider adding the following bound: `'t: 'k`
help: consider adding 'move' keyword before the nested closure
  |
7 |     table.rows.iter().find(|row| row.0 == key).map(move |row| row.1.as_str())
  |                                                    ++++

error[E0597]: `key` does not live long enough
  --> fail_21_rule_first.rs:15:22
   |
14 |         let key = String::from("a"); // a temporary key ...
   |             --- binding `key` declared here
15 |         hit = lookup(&key, &table);
   |                      ^^^^ borrowed value does not live long enough
16 |     } // ... dies here, but the result comes from `table`
   |     - `key` dropped here while still borrowed
17 |     println!("{hit:?}");
   |               ------- borrow later used here
```

It fails *twice*. The correct body is rejected (it returns data with lifetime `'t`), and, because the wrong contract also ties the result to `key`, the perfectly valid caller is rejected as well. The blame would land on the body or on the caller while the real cause, an invisible rule, appears nowhere in the source. The only escape would be to reorder the parameters, which makes argument order carry lifetime meaning. The mirror rule "last parameter" breaks the same way for `fn f(x: &str, y: &str) -> &str { x }`.

Candidate B: "give every input and the output the same lifetime"

This is the most defensible candidate, because it never accepts an unsound program; it merely errs on the strict side. It produces `fn lookup<'a>(key: &'a str, table: &'a Table) -> Option<&'a str>`:

Listing 14 Rule B applied to `lookup` (the body is fine, the caller is not) ✗ rejected

`fail_22_rule_unify.rs` (abridged)

```
// Hypothetical rule B: "give every input and the output the SAME lifetime".
// struct Table and main(): identical to the previous listing

fn lookup<'a>(key: &'a str, table: &'a Table) -> Option<&'a str> {
    table.rows.iter().find(|row| row.0 == key).map(|row| row.1.as_str())
}
```

```
error[E0597]: `key` does not live long enough
  --> fail_22_rule_unify.rs:15:22
   |
14 |         let key = String::from("a"); // a temporary key ...
   |             --- binding `key` declared here
15 |         hit = lookup(&key, &table);
   |                      ^^^^ borrowed value does not live long enough
16 |     } // ... dies here, but the result comes from `table`
   |     - `key` dropped here while still borrowed
17 |     println!("{hit:?}");
   |               ------- borrow later used here
```

The body compiles, but the *same caller* that is valid under the intended contract is now rejected: the result is forced to die with `key`. The error points at the caller's code, in a program in which no lifetime is written anywhere. Only a reader who knows the hidden expansion can understand it, and they would then have to write the explicit signature anyway. Rule B turns the *least* informative contract into a silent default for every function whose result comes from only one of several inputs, which is exactly what Section 3.1 case B looks like in practice. (This is a design trade-off, not an impossibility. Rust chose to make you write the relationship down.)

Candidate C: "infer the contract from the body"

Here the signature would no longer be the contract; the body would. Consider two versions of one library function whose *only* difference is which argument the body returns, with identical caller code:

Listing 15 Version 1: the body returns x ✓ compiles

ok_23_contract_v1.rs

```rust
// Hypothetical rule C: "infer the contract from the body".
// Version 1 of a library function returns `x`; the contract is therefore:
fn pick<'a>(x: &'a str, _y: &str) -> &'a str {
    x
}

fn main() {
    let long = String::from("long-lived");
    let r;
    {
        let short = String::from("short");
        r = pick(&long, &short); // caller relies on: result does not borrow `short`
    }
    println!("{r}");
}
```

```
long-lived
```

Listing 16 Version 2: the body returns y; same caller ✗ rejected

fail_23_contract_v2.rs

```rust
// Version 2: only the BODY changed (`x` -> `y`); the source of the signature is
// untouched, but a body-inferred contract would silently become:
fn pick<'a>(_x: &str, y: &'a str) -> &'a str {
    y
}

fn main() {
    let long = String::from("long-lived");
    let r;
    {
        let short = String::from("short");
        r = pick(&long, &short); // identical caller code
    }
    println!("{r}");
}
```

```
error[E0597]: `short` does not live long enough
  --> fail_23_contract_v2.rs:12:25
   |
11 |         let short = String::from("short");
   |             ----- binding `short` declared here
12 |         r = pick(&long, &short); // identical caller code
   |                         ^^^^^^ borrowed value does not live long enough
13 |     }
   |     - `short` dropped here while still borrowed
14 |     println!("{r}");
   |               --- borrow later used here
```

If the contract were inferred from the body, an innocent edit inside a function, with no change to its visible signature, would break callers, possibly in other crates. Explicit signatures make that change visible (and a deliberate API decision). The rule also cannot work wherever there is no body to inspect:

Listing 17 A trait method declaration has no body ✗ rejected

fail_24_trait_no_body.rs

```rust
// A trait method declaration has no body, so there is nothing to infer from.
trait Pick {
    fn pick(x: &str, y: &str) -> &str;
}

fn main() {}
```

```
error[E0106]: missing lifetime specifier
 --> fail_24_trait_no_body.rs:3:34
  |
3 |     fn pick(x: &str, y: &str) -> &str;
  |                ----     ----     ^ expected named lifetime parameter
  |
  = help: this function's return type contains a borrowed value, but the signature does not say whether it is borrowed from argument 1
  ↪ or argument 2
help: consider introducing a named lifetime parameter
  |
3 |     fn pick<'a>(x: &'a str, y: &'a str) -> &'a str;
  |            ++++     ++          ++          ++
```

The same holds for function pointers, `dyn Fn` signatures and `extern` declarations.

Candidate rule	What it gives `lookup`	Outcome
A: first reference	result tied to `key`	body *and* caller rejected
B: unify everything	all four lifetimes equal	body ok; valid caller rejected
C: infer from body	depends on the body	edits break callers; no help for traits
Rust today: elide only when forced or by `&self`	—	you write `'t` yourself (Listing 12)

The underlying reason

A lifetime in a signature records *which input the output is derived from.* That is a fact about the function's intent, and unlike a type it cannot be recovered from parameter names or positions. The three elision rules cover only the situations where the answer is forced (a single input) or by strong convention (the method's `&self`). Anywhere else, every mechanical guess encodes a claim about the program that is false for some perfectly reasonable function, so the language asks the author.

4 Subtyping and variance

Regions are ordered by *outlives*. If `'a: 'b`, then a reference valid for `'a` may be used where one valid for the shorter `'b` is expected: `&'a T <: &'b T`. Thus `&'static T` is the most specific reference type, and the compiler silently *shrinks* lifetimes at use sites.

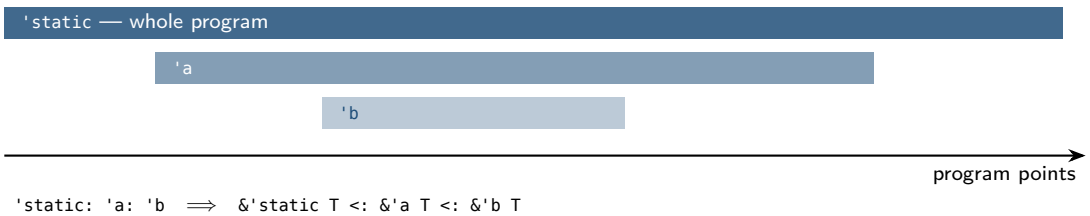


Figure 2: Outlives order on regions induces subtyping on shared references.

How a type constructor propagates this order is its *variance* in each parameter:

Type	in the lifetime	in `T`
`&'a T`	covariant	covariant
`&'a mut T`	covariant	**invariant**
`Box<T>`, `Vec<T>`, `Option<T>`	—	covariant
`Cell<T>`, `RefCell<T>`, `UnsafeCell<T>`	—	**invariant**
`*const T` / `*mut T`	—	covariant / **invariant**
`fn(T) -> U`	—	**contravariant** in `T`, covariant in `U`
`dyn Trait + 'a`	covariant	(trait's own parameters: invariant)

Listing 18 Covariance: `&'static str` shrinks to fit ✓ compiles

ok_13_covariance.rs

```rust
fn pick<'a>(a: &'a str, _b: &'a str) -> &'a str {
    a
}

fn main() {
    let s: &'static str = "static";
    let local = String::from("local");
    // &'static str is silently shrunk to the (shorter) lifetime of `local`.
    println!("{}", pick(s, &local));
}
```

Listing 19 Covariance carries through `Box` ✓ compiles

ok_17_box_covariance.rs

```rust
fn main() {
    let b: Box<&'static str> = Box::new("static");
    let local = String::from("local");
    let mut shorter: Box<&str> = b; // Box<&'static str> -> Box<&'short str>: fine
    shorter = Box::new(&local);
    println!("{shorter}");
}
```

Invariance is the safety net for *writes*. If `&mut &'a str` were covariant you could overwrite a long-lived slot with a short-lived reference through a view that has forgotten the real lifetime:

Listing 20 `&mut T` is invariant in `T` ✗ rejected

fail_13_invariance.rs

```rust
fn overwrite<'a>(slot: &mut &'a str, val: &'a str) {
    *slot = val;
}

fn main() {
    let mut s: &'static str = "static";
    {
        let local = String::from("local");
        overwrite(&mut s, &local); // would leave `s` dangling
    }
    println!("{s}");
}
```

```
error[E0597]: `local` does not live long enough
  --> fail_13_invariance.rs:9:27
   |
6  |     let mut s: &'static str = "static";
   |                ------------ type annotation requires that `local` is borrowed for `'static`
7  |     {
8  |         let local = String::from("local");
   |             ----- binding `local` declared here
9  |         overwrite(&mut s, &local); // would leave `s` dangling
   |                           ^^^^^^ borrowed value does not live long enough
10 |     }
   |     - `local` dropped here while still borrowed
```

The same holds for interior mutability:

Listing 21 `Cell<T>` is invariant in `T` ✗ rejected

fail_17_cell_invariance.rs

```rust
use std::cell::Cell;

fn main() {
    let c: Cell<&'static str> = Cell::new("static");
    let local = String::from("local");
    let r: &Cell<&str> = &c; // Cell<&'static str> is NOT a Cell<&'short str>
    r.set(&local);
}
```

```
error[E0597]: `local` does not live long enough
 --> fail_17_cell_invariance.rs:7:11
  |
4 |     let c: Cell<&'static str> = Cell::new("static");
  |            ------------------ type annotation requires that `local` is borrowed for `'static`
5 |     let local = String::from("local");
  |         ----- binding `local` declared here
6 |     let r: &Cell<&str> = &c; // Cell<&'static str> is NOT a Cell<&'short str>
7 |     r.set(&local);
  |           ^^^^^^ borrowed value does not live long enough
8 | }
  | - `local` dropped here while still borrowed
```

For the type-theoretically inclined: the lifetimes form a preorder under *outlives*; each type constructor is a (co-, contra- or non-variant) functor on it. `&mut T` places `T` in both a read (covariant) and a write (contravariant) position, and the two together force invariance. `'static` is the greatest element of the order.

5 The anonymous lifetime '_

5.1 What it means, precisely

'_ is *not a lifetime name*: it cannot be referred to twice. Every occurrence introduces its own, independent lifetime, and *what that lifetime is* depends on where it appears:

Position	Meaning of '_	Example
Function *parameter* type	A fresh lifetime parameter, identical to eliding it	`x: &'_ str, r: Ref<'_, T>`
Function *return* type	Whichever input lifetime elision rules 2/3 select; E0106 if none	`-> Iter<'_, T>`
`impl` header	A fresh parameter, unnameable in the body	`impl Display for Parser<'_>`
After + on `dyn`/`impl` in a signature	The same elision choice as the return type	`Box<dyn Fn() + '_>, impl Iterator + '_`

Position	Meaning of '_	Example
Type in a *body* (`let`, turbofish)	An inference variable	`let g: MutexGuard<'_, T> = ..`
Struct/enum field	*Not allowed* (E0106)	`struct S { r: &'_ str }`

Key idea

Read '_ as "a lifetime exists here, I don't need to name it". Two occurrences are **never** forced to be equal: `fn f(a: &'_ str, b: &'_ str)` is `fn f<'x, 'y>(a: &'x str, b: &'y str)`, not `fn f<'a>(a: &'a str, b: &'a str)`.

5.2 Job 1 — make hidden lifetime parameters visible

Many std types carry a lifetime (`Chars<'a>`, `Ref<'b, T>`, `fmt::Formatter<'_>`…). You may omit it, but then a borrow is hidden in the signature. The lint `elided_lifetimes_in_paths` (part of `rust_2018_idioms`) flags it, and '_ is the fix:

Listing 22 `SplitWhitespace<'_>` vs. the hidden form ✓ compiles

ok_04_underscore_return.rs

```rust
#![warn(elided_lifetimes_in_paths)]
use std::str::SplitWhitespace;

// Explicit: the hidden lifetime parameter of SplitWhitespace is visible.
fn words(s: &str) -> SplitWhitespace<'_> {
    s.split_whitespace()
}

// Legal, but the borrow is invisible in the signature (the lint fires).
fn words_hidden(s: &str) -> SplitWhitespace {
    s.split_whitespace()
}

fn main() {
    let t = String::from("one two three");
    println!("{}", words(&t).count() + words_hidden(&t).count());
}
```

```
warning: hidden lifetime parameters in types are deprecated
  --> ok_04_underscore_return.rs:10:29
   |
10 | fn words_hidden(s: &str) -> SplitWhitespace {
   |                             ^^^^^^^^^^^^^^^ expected lifetime parameter
   |
note: the lint level is defined here
  --> ok_04_underscore_return.rs:1:9
   |
1  | #![warn(elided_lifetimes_in_paths)]
   |         ^^^^^^^^^^^^^^^^^^^^^^^^^
help: indicate the anonymous lifetime
   |
10 | fn words_hidden(s: &str) -> SplitWhitespace<'_> {
   |                                            ++++
```

5.3 Job 2 — impl headers, and when you need a real name instead

`impl Display for Parser<'_>` is the same as `impl<'a> Display for Parser<'a>`; prefer the short form whenever the body never mentions the lifetime (see `Display for Parser` in Listing 5 and the `Formatter<'_>` parameter). As soon as you must *refer* to it, e.g. to return `Option<&'src str>`, you need a name, and this is the dividing line between '_ and a named lifetime. `impl Parser<'_> { fn next_word(&mut self) -> Option<&'src str> }` cannot be written.

5.4 Job 3 — opting trait objects and `impl Trait` into borrowing

This is the use case that is least often explained. A trait object in a signature has a hidden lifetime bound, and for `Box<dyn ..>` the default is `'static` (Section 6.5). To let it borrow from a parameter, say so with `'_`:

Listing 23 `Box<dyn Fn(&str) -> String + '_>` ✓ compiles

ok_05_dyn_underscore.rs

```
fn make_prefixer(prefix: &str) -> Box<dyn Fn(&str) -> String + '_> {
    Box::new(move |s| format!("{prefix}{s}"))
}

fn main() {
    let p = String::from(">> ");
    let f = make_prefixer(&p);
    println!("{}", f("hello"));
}
```

Listing 24 Without `+ '_` the closure must be `'static` ✗ rejected

fail_05_dyn_static.rs

```
// `Box<dyn Fn..>` in a signature means `Box<dyn Fn.. + 'static>`.
fn make_prefixer(prefix: &str) -> Box<dyn Fn(&str) -> String> {
    Box::new(move |s| format!("{prefix}{s}"))
}

fn main() {
    let p = String::from(">> ");
    println!("{}", make_prefixer(&p)("hello"));
}
```

```
error: lifetime may not live long enough
 --> fail_05_dyn_static.rs:3:5
  |
2 | fn make_prefixer(prefix: &str) -> Box<dyn Fn(&str) -> String> {
  |                          - let's call the lifetime of this reference `'1`
3 |     Box::new(move |s| format!("{prefix}{s}"))
  |     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ returning this value requires that `'1` must outlive `'static`
  |
help: to declare that the trait object captures data from argument `prefix`, you can add an explicit `'_` lifetime bound
  |
2 | fn make_prefixer(prefix: &str) -> Box<dyn Fn(&str) -> String + '_> {
  |                                                            ++++
```

Note that the compiler itself suggests `'_`. The same applies to `impl Trait` in return position. In edition 2021 an opaque type captures *type* parameters but *not* the lifetime of an elided reference argument unless you say so:

Listing 25 `impl Iterator + '_` ✓ compiles

ok_06_rpit.rs

```
fn chars_of(s: &str) -> impl Iterator<Item = char> + '_ {
    s.chars()
}

fn main() {
    let t = String::from("abc");
    println!("{:?}", chars_of(&t).collect::<Vec<_>>());
}
```

Listing 26 Edition 2021: the hidden type captures a lifetime that is not in the bounds ✗ rejected

fail_06_rpit.rs

```
// Edition 2021: `impl Trait` does not capture the elided lifetime of `s`.
fn chars_of(s: &str) -> impl Iterator<Item = char> {
    s.chars()
}

fn main() {
    let t = String::from("abc");
    println!("{:?}", chars_of(&t).collect::<Vec<_>>());
}
```

```
error[E0700]: hidden type for `impl Iterator<Item = char>` captures lifetime that does not appear in bounds
 --> fail_06_rpit.rs:3:5
  |
2 | fn chars_of(s: &str) -> impl Iterator<Item = char> {
  |                ----     -------------------------- opaque type defined here
  |                |
  |                hidden type `Chars<'_>` captures the anonymous lifetime defined here
3 |     s.chars()
  |     ^^^^^^^^
  |
help: to declare that `impl Iterator<Item = char>` captures `'_`, you can add an explicit `'_` lifetime bound
  |
2 | fn chars_of(s: &str) -> impl Iterator<Item = char> + '_ {
  |                                                    ++++
```

Edition note

Rust 2024 (stable since 1.85) changes return-position `impl Trait` to capture *all* in-scope lifetimes by default, so Listing 26 compiles there, and `+ use<..>` (precise capturing, 1.82) lets you opt *out*. I verified the 2021 behaviour above with rustc 1.75; the 2024 statement is from my own knowledge of the edition, was not checked by this document's build, and is worth confirming on your toolchain.

5.5 What '_ cannot do

'_ does not resolve ambiguity. When elision has nothing to choose, writing '_ in the output changes nothing:

Listing 27 '_ does not make an ambiguous signature legal ✗ rejected

fail_04_ambiguous.rs

```
fn pick(a: &str, b: &str) -> &str { a }
fn pick_underscore(a: &str, b: &str) -> &'_ str { a }

fn main() {
    println!("{} {}", pick("x", "y"), pick_underscore("x", "y"));
}
```

```
error[E0106]: missing lifetime specifier
 --> fail_04_ambiguous.rs:1:30
  |
1 | fn pick(a: &str, b: &str) -> &str { a }
  |            ----     ----     ^ expected named lifetime parameter
  |
  = help: this function's return type contains a borrowed value, but the signature does not say whether it is borrowed from `a` or `b`
help: consider introducing a named lifetime parameter
  |
1 | fn pick<'a>(a: &'a str, b: &'a str) -> &'a str { a }
  |        ++++     ++          ++          ++

error[E0106]: missing lifetime specifier
 --> fail_04_ambiguous.rs:2:42
  |
2 | fn pick_underscore(a: &str, b: &str) -> &'_ str { a }
  |                       ----     ----      ^^ expected named lifetime parameter
  |
  = help: this function's return type contains a borrowed value, but the signature does not say whether it is borrowed from `a` or `b`
help: consider introducing a named lifetime parameter
  |
2 | fn pick_underscore<'a>(a: &'a str, b: &'a str) -> &'a str { a }
  |                   ++++     ++          ++          ~~
```

It also does not mean "infer it from the body". Signature lifetimes are *never* inferred from the body; only body-level types (`let x: Ref<'_, T>`) use `'_` as an inference placeholder.

Checklist: which of `'a`, `'_`, or nothing?

1. The lifetime appears once and I never mention it again ⇒ `'_` (or elide it).
2. I need to tie two positions together (two inputs, or an input and the output) ⇒ a *named* lifetime.
3. I want the output to follow the `&self` borrow ⇒ elide, or write `'_`.
4. I want the output to follow a *field's* lifetime rather than `&self` ⇒ name the struct's parameter.
5. A trait object/`impl Trait` in a return type should borrow from an argument ⇒ add `+ '_` or `+ 'a`.

6 The 'static lifetime

6.1 Two meanings that are constantly conflated

Form	What it is	What it says
`&'static T`	a *reference type*	the pointee is valid for the remainder of the program (string literals, `static` items, leaked allocations)
`T: 'static`	a *bound* on a type	every lifetime inside `T` is `'static`, i.e. `T` contains no borrow that can expire. Owning types satisfy it

Formally, `&'a U: 'static` holds iff `'a` is `'static` (and `U: 'static`); `Foo<'a>: 'static` holds iff `'a` is `'static`; a concrete type with no lifetime parameter anywhere inside it (such as `String`) satisfies the bound unconditionally.

6.2 Where &'static references come from

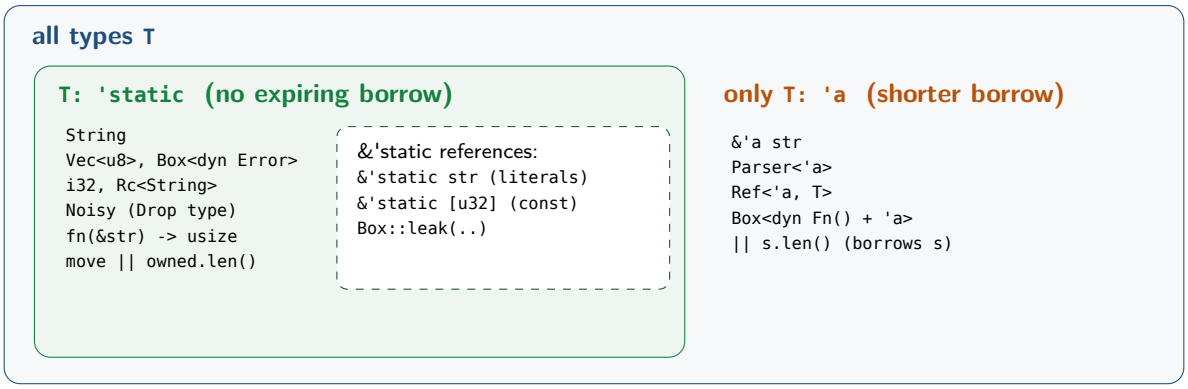


Figure 3: `T: 'static` is a property of the *type*; `&'static T` is a small subset of it.

Listing 28 Four sources of `&'static` ✓ compiles

ok_07_static_sources.rs

```rust
static BANNER: &str = "lifetimes";        // type is &'static str
const LIMITS: &[u32] = &[1, 2, 3];        // type is &'static [u32]

fn promoted() -> &'static i32 {
    &42                                   // constant promotion
}

fn leaked(s: String) -> &'static str {
    Box::leak(s.into_boxed_str())         // heap data, never freed
}

fn main() {
    let l: &'static str = leaked(format!("{}-{}", "dyn", 7));
    println!("{BANNER} {LIMITS:?} {} {l}", promoted());
}
```

```
lifetimes [1, 2, 3] 42 dyn-7
```

- **Literals** live in the binary's read-only data. `static` and `const` items of reference type get `'static` by the item default rule.
- **Constant promotion**: `&42` or `&[1, 2, 3]` in a function body is promoted to a static when the expression is constant and has no destructor or interior mutability. `&String::new()` is *not* promoted.
- **Leaking**: `Box::leak` returns `&'a mut T` for any `'a` you ask for, including `'static`. It is a deliberate memory leak: fine for process-lifetime singletons and interned strings, a bug when done per request.
- **Globals** (`static` + `OnceLock`/`LazyLock`) hand out `'static` references to lazily initialised data.

6.3 `T: 'static` does *not* mean "lives forever"

The bound constrains the *type*, not the *value's* lifetime. A value of a `'static` type can be created and dropped in the same statement:

Listing 29 A 'static type that is dropped immediately ✓ compiles

ok_08_static_bound.rs

```rust
struct Noisy(&'static str);

impl Drop for Noisy {
    fn drop(&mut self) {
        println!("drop {}", self.0);
    }
}

fn requires_static<T: 'static>(_t: T) {
    println!("inside");
}

fn main() {
    requires_static(Noisy("a")); // Noisy: 'static -- yet it dies right here
    println!("after");
    requires_static(String::from("owned")); // local, owned: fine
    requires_static(7u8);
    requires_static("literal");
}
```

```
inside
drop a
after
inside
inside
inside
```

Listing 30 Only a *borrow* of a local fails the bound ✗ rejected

fail_08_static_bound.rs

```rust
fn requires_static<T: 'static>(_t: T) {}

fn main() {
    let local = String::from("x");
    requires_static(&local); // &'a String: 'static  only if 'a == 'static
}
```

```
error[E0597]: `local` does not live long enough
 --> fail_08_static_bound.rs:5:21
  |
4 |     let local = String::from("x");
  |         ----- binding `local` declared here
5 |     requires_static(&local); // &'a String: 'static  only if 'a == 'static
  |     ----------------^^^^^^-
  |     |               |
  |     |               borrowed value does not live long enough
  |     argument requires that `local` is borrowed for `'static`
6 | }
  | - `local` dropped here while still borrowed
```

So the practical test is "does the type contain a borrow that can expire?" (every "yes" row compiles in `ok_15`; every "no" row is rejected in `fail_15`):

Type / value	: 'static?	Why
`String`, `Vec<String>`, `i32`	yes	owns all its data
`&'static str`	yes	the only lifetime inside is `'static`
`Rc<String>`	yes	iff the contents are
`Box<dyn Error>`	yes	default object bound is `'static`
`fn(&str)`, fn items	yes	no captured state
`move` closure owning a `String`	yes	captures by value
`&'a str` (local `'a`)	no	short borrow inside
`Parser<'a>`	no	lifetime parameter is not `'static`
non-`move` closure using a local	no	captures a borrow

6.4 Why APIs ask for `T: 'static`

Threads and tasks. `thread::spawn` (and async runtimes' `spawn`) hand the closure to something that may outlive the caller's stack frame, so it cannot contain borrows of that frame. Either move ownership in, or use *scoped* threads, which are joined before the borrowed data can die and therefore need no `'static`:

Listing 31 Own it, or borrow it with `thread::scope` ✓ compiles

ok_09_threads.rs

```rust
use std::thread;

fn main() {
    let data = vec![1, 2, 3];

    // 1. Ownership moves in: the closure is 'static.
    let owned = data.clone();
    thread::spawn(move || println!("moved: {owned:?}")).join().unwrap();

    // 2. Borrowing: scoped threads are joined before the scope ends.
    thread::scope(|s| {
        s.spawn(|| println!("sum = {}", data.iter().sum::<i32>()));
        s.spawn(|| println!("len = {}", data.len()));
    });
    println!("still mine: {data:?}");
}
```

```
moved: [1, 2, 3]
sum = 6
len = 3
still mine: [1, 2, 3]
```

Listing 32 Borrowing a local into `thread::spawn` ✗ rejected

fail_09_spawn_borrow.rs

```rust
use std::thread;

fn main() {
    let data = vec![1, 2, 3];
    let h = thread::spawn(|| println!("{data:?}")); // borrows `data`
    h.join().unwrap();
}
```

```
error[E0373]: closure may outlive the current function, but it borrows `data`, which is owned by the current function
 --> fail_09_spawn_borrow.rs:5:27
  |
5 |     let h = thread::spawn(|| println!("{data:?}")); // borrows `data`
  |                           ^^            ---- `data` is borrowed here
  |                           |
  |                           may outlive borrowed value `data`
  |
note: function requires argument type to outlive `'static`
 --> fail_09_spawn_borrow.rs:5:13
  |
5 |     let h = thread::spawn(|| println!("{data:?}")); // borrows `data`
  |             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
help: to force the closure to take ownership of `data` (and any other referenced variables), use the `move` keyword
  |
5 |     let h = thread::spawn(move || println!("{data:?}")); // borrows `data`
  |                           ++++
```

`Any` and downcasting. `Any` requires `'static` because `TypeId` is computed *after* lifetimes are erased: `&'a T` and `&'b T` would be indistinguishable, and downcasting could then forge lifetimes.

Trait objects stored for a long time. `Box<dyn Error>`, callbacks in registries and `Arc<dyn Trait>` fields all default to `'static`.

Generic propagation. If your generic function calls any of the above, the bound must appear on *your* `T` too:

Listing 33 E0310: the bound has to be written on the generic parameter ✗ rejected

fail_14_E0310.rs

```rust
use std::thread;

fn spawn_drop<T: Send>(t: T) {
    thread::spawn(move || drop(t));
}

fn main() {
    spawn_drop(1);
}
```

```
error[E0310]: the parameter type `T` may not live long enough
 --> fail_14_E0310.rs:4:5
  |
4 |     thread::spawn(move || drop(t));
  |     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  |     |
  |     the parameter type `T` must be valid for the static lifetime...
  |     ...so that the type `T` will meet its required lifetime bounds
  |
help: consider adding an explicit lifetime bound
  |
3 | fn spawn_drop<T: Send + 'static>(t: T) {
  |                       +++++++++
```

Listing 34 Adding `+ 'static` fixes it ✓ compiles

ok_14_E0310_fixed.rs

```rust
use std::thread;

fn spawn_drop<T: Send + 'static>(t: T) {
    thread::spawn(move || drop(t)).join().unwrap();
}

fn main() {
    spawn_drop(String::from("ok"));
}
```

6.5 Defaults for trait-object lifetime bounds

When you write `dyn Trait` without a lifetime bound, the compiler picks one from the surrounding type:

You write	It means	Rule
`Box<dyn T>`	`Box<dyn T + 'static>`	container has no lifetime
`Rc<dyn T>`, `Arc<dyn T>`	`... dyn T + 'static`	same
`&'a dyn T`	`&'a (dyn T + 'a)`	bounded by the reference
`&'a mut dyn T`	`&'a mut (dyn T + 'a)`	same
`Ref<'a, dyn T>`	`Ref<'a, dyn T + 'a>`	parameter has `T: 'a`
`trait T: 'x`, then `dyn T`	`dyn T + 'x`	trait's own bound
`dyn T` inside a fn *body*	inferred	expression context

This asymmetry is the root of the infamous "*returning this value requires that `'1` must outlive `'static`*" error. The complete program below names the bound; deleting `+ 'a` reproduces the error (second listing).

Listing 35 Name the bound where the object must borrow ✓ compiles

ok_10_object_defaults.rs

```
trait Shape {
    fn area(&self) -> f64;
}

struct Circle<'a> {
    r: &'a f64,
}

impl Shape for Circle<'_> {
    fn area(&self) -> f64 {
        3.14159 * self.r * self.r
    }
}

// Must say `+ 'a`: the default for Box<dyn Shape> would be 'static.
fn boxed<'a>(r: &'a f64) -> Box<dyn Shape + 'a> {
    Box::new(Circle { r })
}

// Here the default is already right: &'x [&'y (dyn Shape + 'y)]
fn total(shapes: &[&dyn Shape]) -> f64 {
    shapes.iter().map(|s| s.area()).sum()
}

fn main() {
    let r = 2.0;
    let b = boxed(&r);
    let c = Circle { r: &r };
    println!("{:.2} {:.2}", b.area(), total(&[&c, &*b]));
}
```

```
12.57 25.13
```

Listing 36 Plain Box<dyn Shape> silently means + 'static ✗ rejected

fail_10_boxed_static.rs (abridged)

```
// trait Shape, struct Circle<'a> and its impl: exactly as in the previous listing

fn boxed(r: &f64) -> Box<dyn Shape> {
    Box::new(Circle { r })
}

fn main() {
    let r = 2.0;
    println!("{:.2}", boxed(&r).area());
}
```

```
error: lifetime may not live long enough
  --> fail_10_boxed_static.rs:16:5
   |
15 | fn boxed(r: &f64) -> Box<dyn Shape> {
   |             - let's call the lifetime of this reference `'1`
16 |     Box::new(Circle { r })
   |     ^^^^^^^^^^^^^^^^^^^^^^ returning this value requires that `'1` must outlive `'static`
   |
help: to declare that the trait object captures data from argument `r`, you can add an explicit `'_` lifetime bound
   |
15 | fn boxed(r: &f64) -> Box<dyn Shape + '_> {
   |                                    ++++
```

The same trap hides behind ? and Box<dyn Error>: an error type that *borrows* from its input cannot be converted into the default Box<dyn Error>.

Listing 37 `Box<dyn Error + '_>` (often better still: own the data in the error) ✓ compiles

ok_11_error_box.rs

```rust
use std::error::Error;
use std::fmt;

#[derive(Debug)]
struct BadInput<'a> {
    input: &'a str,
}

impl fmt::Display for BadInput<'_> {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "bad input: {:?}", self.input)
    }
}

impl Error for BadInput<'_> {}

fn parse(s: &str) -> Result<i32, Box<dyn Error + '_>> {
    s.parse::<i32>().map_err(|_| BadInput { input: s }.into())
}

fn main() {
    let text = String::from("12x");
    let res = parse(&text);
    match res {
        Ok(n) => println!("{n}"),
        Err(e) => println!("{e}"),
    }
}
```

```
bad input: "12x"
```

Listing 38 Default `Box<dyn Error>` demands `'static` ✗ rejected

fail_11_error_box.rs (abridged)

```rust
// BadInput<'a> and its Display / Error impls: exactly as in the previous listing

fn parse(s: &str) -> Result<i32, Box<dyn Error>> {
    s.parse::<i32>().map_err(|_| BadInput { input: s }.into())
}

fn main() {
    println!("{:?}", parse("12x").is_err());
}
```

```
error: lifetime may not live long enough
  --> fail_11_error_box.rs:18:5
   |
17 | fn parse(s: &str) -> Result<i32, Box<dyn Error>> {
   |             - let's call the lifetime of this reference `'1`
18 |     s.parse::<i32>().map_err(|_| BadInput { input: s }.into())
   |     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ returning this value requires that `'1` must outlive `'static`
   |
help: to declare that the trait object captures data from argument `s`, you can add an explicit `'_` lifetime bound
   |
17 | fn parse(s: &str) -> Result<i32, Box<dyn Error + '_>> {
   |                                                ++++
help: consider adding 'move' keyword before the nested closure
   |
18 |     s.parse::<i32>().map_err(move |_| BadInput { input: s }.into())
   |                              ++++
```

6.6 Myths and anti-patterns

Myth / habit	Reality
"`T: 'static` means the value lives for the whole program."	It restricts the *type*. A `String` local to one function is `'static`; it is dropped normally (Listing 29).
"The compiler says `'static` so I'll add `&'static` to the parameter."	You have just forced every caller to supply literals or leaked data. Usually the error means a trait-object or spawn bound (see above); fix *that*.
"Leak it; it's only a small `String`."	`Box::leak` per call is an unbounded memory leak. Acceptable for one-time configuration, interned names, and test fixtures only.
"`struct Cfg { name: &'static str }` is flexible."	It accepts only literals and leaked strings. Prefer `String`, `Arc<str>`, a lifetime parameter `Cfg<'a>`, or `Cow<'static, str>` when most values are literals.
"`&'static` and `T: 'static` are the same thing."	One is a reference whose pointee lives forever, the other a closure under owned data (Figure 3).

A decision ladder for "I need this to be `'static`"

1. **Own it**: clone, `String`, `Vec`, or share with `Arc`.
2. **Scope it**: `thread::scope`, a borrowing struct `Foo<'a>`, a bound like `dyn Trait + 'a`.
3. **Mix**: `Cow<'a, str>` or `Cow<'static, str>`.
4. **Leak or global**: `Box::leak`, `OnceLock`, `LazyLock` — only for genuine process-lifetime data.

7 Higher-ranked bounds: `for<'a>`

A bound such as `F: Fn(&str) -> &str` is sugar for `F: for<'a> Fn(&'a str) -> &'a str`: "works for *every* lifetime", not for one lifetime fixed by the caller. This is what lets `f` be called on a string created inside the function. You write `for<'a>` explicitly only when the lifetime appears somewhere the sugar does not reach, for example on a reference to a generic parameter:

Listing 39 Explicit `for<'a>` on `&'a C` ✓ compiles

ok_12_hrtb.rs

```
// F must work for *every* lifetime 'a, not one fixed lifetime.
fn apply<F>(f: F, s: &str) -> usize
where
    F: for<'a> Fn(&'a str) -> &'a str,
{
    f(s).len()
}

// A bound that genuinely needs `for<'a>`: `&C` must be iterable for any 'a.
fn sum_all<C>(c: &C) -> i32
where
    for<'a> &'a C: IntoIterator<Item = &'a i32>,
{
    c.into_iter().sum()
}

fn main() {
    println!("{}", apply(|s| s.trim(), "  hi  "));
    println!("{}", sum_all(&vec![1, 2, 3]));
}
```

```
2
6
```

8 Error catalogue

Code	Message	Usually means	Typical fix
E0106	missing lifetime specifier	Elision has no unique answer, or a field has no lifetime	Name a lifetime; add `'a` to the struct
E0261	use of undeclared lifetime name	`'a` used but not declared in `<..>`	Declare `<'a>` on the fn/impl/struct
E0597	`x` does not live long enough	A reference outlives its referent; also fires for `T: 'static` arguments	Shorten the use, move/clone the value, or restructure scopes
E0499	cannot borrow as mutable more than once	Often an elided `&mut self` lifetime pinning results	Return the struct's own lifetime (Listing 5)
E0502	mutable borrow while immutably borrowed	A shared borrow is still live at the conflicting access	Reorder, end the borrow earlier, or clone
E0505	move out while borrowed	Moving a value that a live borrow points to	Borrow after the move or clone
E0373	closure may outlive the current function	Non-`move` closure given to `spawn`-like API	`move`, or `thread::scope`
E0310	parameter type `T` may not live long enough	Generic code needs `T: 'static` (or `T: 'a`)	Add the bound on the parameter
E0700	hidden type captures lifetime not in bounds	`impl Trait` in edition 2021 hides a borrow	Add `+ '_` / `+ 'a`; or edition 2024
E0716	temporary value dropped while borrowed	Borrowing a temporary beyond its statement	Bind it to a `let`
E0521	borrowed data escapes outside of function	A short borrow flows into a longer-lived place	Own the data, or add an explicit lifetime relation
(none)	lifetime may not live long enough	Region constraint violated between two signature lifetimes; often `'1: 'static` from a default object bound	Compare the lifetimes the message names; add `'_`/`'a` bounds

Codes E0106, E0261, E0597, E0499, E0502, E0373, E0310 and E0700 appear in programs verified in this document. E0505, E0716 and E0521 are listed from general knowledge of the compiler.

9 Quick reference

I want to...	Write
return a reference derived from the single input	elide: `fn f(x: &T) -> &U`
return a view/iterator over `&self`	`-> Iter<'_, T>` or `-> impl Iterator<Item = &T> + '_`
return data from a struct field's referent, not the `&mut self` borrow	`impl<'a> S<'a> { fn f(&mut self) -> &'a T }`
tie two parameters to the result	`fn f<'a>(x: &'a T, y: &'a T) -> &'a T`
implement a trait for a borrowing type	`impl Trait for Wrapper<'_>` (name it only if used)
return a closure/trait object that captures an argument	`Box<dyn Fn(..) -> .. + '_>`
store a trait object in a long-lived struct	`Box<dyn Trait>` (`'static`) or `struct S<'a> { b: Box<dyn Trait + 'a> }`
run a closure on another thread with borrowed data	`thread::scope(|s| { s.spawn(|| ..); })`
make generic code spawnable	`T: Send + 'static`
return an error that borrows its input	`Result<_, Box<dyn Error + '_>>`, or own the data in the error
accept a callback that handles any borrow	`F: for<'a> Fn(&'a str) -> &'a str` (or `Fn(&str) -> &str`)

Ten rules that cover most real code

1. A lifetime is a region of program points, not a duration; annotations never extend one.
2. Only signatures need lifetimes; bodies are inferred.
3. Elision picks the single input, else `&self`, else fails.
4. The `&mut self` borrow and a struct's own lifetime are different lifetimes; return the one you mean.
5. `'_` = "a lifetime I don't need to name"; never "the same as the other one".
6. `'_` after `+` on a trait object or `impl Trait` opts it into borrowing from the elided input.
7. `&mut T` and `Cell<T>` are invariant in `T`; shared references are covariant.
8. `&'static T` is a reference; `T: 'static` is a bound that owned types satisfy.
9. `Box<dyn Trait>` means `+ 'static`; `&'a dyn Trait` means `+ 'a`.
10. When tempted to add `'static`, first try owning, scoping, or `Cow`.

A Verification log

Each program was compiled with `rustc 1.75.0 --edition 2021` (Ubuntu package). "Expected" follows the file prefix: `ok_` must compile, `fail_` must be rejected. The last column is the first diagnostic of the real run.

Program	Expected	Result	First diagnostic
`fail_01_dangling.rs`	rejected	pass	`error[E0597]: \`inner\` does not live long enough`
`ok_01_longest.rs`	compiles	pass	—
`fail_02_nll.rs`	rejected	pass	`error[E0502]: cannot borrow \`v\` as mutable because it is also borrowed as immutable`
`ok_02_nll.rs`	compiles	pass	—
`fail_03_elided_self.rs`	rejected	pass	`error[E0499]: cannot borrow \`p\` as mutable more than once at a time`
`ok_03_parser.rs`	compiles	pass	—
`fail_04_ambiguous.rs`	rejected	pass	`error[E0106]: missing lifetime specifier (+1 more)`
`ok_04_underscore_return.rs`	compiles	pass	`warning: hidden lifetime parameters in types are deprecated`
`fail_05_dyn_static.rs`	rejected	pass	`error: lifetime may not live long enough`
`ok_05_dyn_underscore.rs`	compiles	pass	—
`fail_06_rpit.rs`	rejected	pass	`error[E0700]: hidden type for \`impl Iterator<Item = char>\` captures lifetime that doe...`
`ok_06_rpit.rs`	compiles	pass	—
`ok_07_static_sources.rs`	compiles	pass	—
`fail_08_static_bound.rs`	rejected	pass	`error[E0597]: \`local\` does not live long enough`
`ok_08_static_bound.rs`	compiles	pass	—
`fail_09_spawn_borrow.rs`	rejected	pass	`error[E0373]: closure may outlive the current function, but it borrows \`data\`, which ...`
`ok_09_threads.rs`	compiles	pass	—
`fail_10_boxed_static.rs`	rejected	pass	`error: lifetime may not live long enough`
`ok_10_object_defaults.rs`	compiles	pass	—
`fail_11_error_box.rs`	rejected	pass	`error: lifetime may not live long enough`
`ok_11_error_box.rs`	compiles	pass	`error[E0597]: \`text\` does not live long enough`
`ok_12_hrtb.rs`	compiles	pass	—
`fail_13_invariance.rs`	rejected	pass	`error[E0597]: \`local\` does not live long enough`
`ok_13_covariance.rs`	compiles	pass	—
`fail_14_E0310.rs`	rejected	pass	`error[E0310]: the parameter type \`T\` may not live long enough`
`ok_14_E0310_fixed.rs`	compiles	pass	—
`fail_15_static_table_no.rs`	rejected	pass	`error[E0597]: \`s\` does not live long enough (+2 more)`
`ok_15_static_table.rs`	compiles	pass	—
`fail_16_struct_field_elided.rs`	rejected	pass	`error[E0106]: missing lifetime specifier (+2 more)`
`fail_17_cell_invariance.rs`	rejected	pass	`error[E0597]: \`local\` does not live long enough`
`ok_17_box_covariance.rs`	compiles	pass	—
`fail_18_elision_cases.rs`	rejected	pass	`error[E0106]: missing lifetime specifier (+1 more)`
`ok_18_elision_fixes.rs`	compiles	pass	—
`fail_19_struct_single_ref.rs`	rejected	pass	`error[E0106]: missing lifetime specifier`
`ok_19_struct_fixes.rs`	compiles	pass	—
`fail_20_static_field_local.rs`	rejected	pass	`error[E0597]: \`text\` does not live long enough`
`fail_21_rule_first.rs`	rejected	pass	`error: lifetime may not live long enough (+1 more)`
`ok_21_lookup_explicit.rs`	compiles	pass	—
`fail_22_rule_unify.rs`	rejected	pass	`error[E0597]: \`key\` does not live long enough`
`fail_23_contract_v2.rs`	rejected	pass	`error[E0597]: \`short\` does not live long enough`
`ok_23_contract_v1.rs`	compiles	pass	—
`fail_24_trait_no_body.rs`	rejected	pass	`error[E0106]: missing lifetime specifier`

B Further reading

Titles only; I could not open or check any of these while writing, so please confirm sections and current wording yourself: *The Rust Programming Language* (generic types, traits and lifetimes chapter); *The Rust Reference* (lifetime elision, trait object lifetime bounds, subtyping and variance); *The Rustonomicon* (lifetimes, subtyping and variance, higher-rank trait bounds); RFC 2094 (non-lexical lifetimes).